ツール利用型 AI エージェントの自動レッドチーミング基盤構築に向けた適応的探索手法の検討

東 拓矢[1,a)]　中井 綱人[1]　大西 健斗[1]　泰山 幸大[1]

概要：AI エージェントの実用化に伴い，攻撃者の観点から安全性を検証するレッドチーミングの重要性が高まっている．特に，外部ツールを自律的に呼び出す AI エージェントでは，不適切な操作が外部システムやデータに影響を及ぼしうるため，継続的な検証が必要である．しかし，多様な攻撃の設計，実行，評価を人手で繰り返すには多大なコストを要し，自動レッドチーミング基盤が求められる．その構築では，対象モデルによってツール呼び出しの成否や出力挙動が異なることに加え，探索時に発見した攻撃候補が再実行時にも同じ挙動を示すとは限らない．さらに，攻撃の試行と評価には時間を要するため，限られた時間内に多様な攻撃を十分に検証することは難しい．反復的な安全性検証を可能にするには，安全性違反の重大度と実行時間の両方を考慮した探索が必要となる．本稿では，各攻撃方針の試行結果から安全性違反の重大度と実行時間を評価し，より重大な安全性違反を短時間で引き起こした方針を以降の探索で優先的に選択する適応的探索手法を提案する．Kaggle の「AI Agent Security – Multi-Step Tool Attacks」コンペティションを題材とし，GPT-OSS および Gemma 4 を用いて公開ガードレール環境で実験した．同一の実行時間制約の下で，コンペティションの評価式に基づくローカル再実行スコアを比較した結果，提案手法は一つの攻撃方針のみを用いる固定方式より高い値を示した．以上より，モデルの挙動と実行時間に基づく複数方針の適応的選択が，限られた時間内での有効な攻撃候補の発見に寄与する可能性を示した．

キーワード：ツール利用型 AI エージェント，AI エージェントセキュリティ

Adaptive Search toward Automated Red Teaming of Tool-Using AI Agents

Takuya Higashi[1,a)]　Tsunato Nakai[1]　Kento Onishi[1]　Kodai Taiyama[1]

Abstract: As AI agents become increasingly deployed in practical applications, red teaming—evaluating system safety from an adversarial perspective—has become increasingly important. In particular, AI agents that autonomously invoke external tools require continuous testing because inappropriate actions may affect external systems and data. However, manually repeating the design, execution, and evaluation of diverse attacks is costly, motivating the development of automated red-teaming frameworks. Building such frameworks is challenging because the success of tool invocation and the resulting behavior vary across target models, and attack candidates discovered during exploration may not reproduce the same behavior when replayed. Moreover, because executing and evaluating attacks is time-consuming, it is difficult to test a sufficiently diverse set of attacks within a limited time budget. Iterative safety testing therefore requires a search method that considers both the severity of safety violations and the time required for testing. We propose an adaptive search method that evaluates the severity of safety violations and execution time observed for each attack strategy, and preferentially selects strategies that induce more severe violations in less time during subsequent exploration. Using the Kaggle “AI Agent Security – Multi-Step Tool Attacks” competition as an evaluation setting, we conducted experiments with GPT-OSS and Gemma 4 under a public guardrail. Quantitative evaluation under identical execution-time constraints showed that the proposed method achieved a higher local replay score, computed using the competition scoring formula, than a fixed baseline using a single attack strategy. These results suggest that adaptively selecting among multiple attack strategies based on model behavior and execution time can facilitate the discovery of effective attack candidates within a limited time budget.

Keywords: Tool-Using AI Agents, AI Agent Security

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

AI エージェントの実用化に伴い，攻撃者の観点からシステムの安全性を検証するレッドチーミングの重要性が高まっている．特に，大規模言語モデル (LLM) を中核として外部ツールを自律的に呼び出すツール利用型 AI エージェントは，文章生成に加えて，ウェブ検索，ファイル操作，電子メール送信，外部通信などを含む複数段階の処理を実行できる [1], [2]．一方，モデルの誤判断や悪意ある入力の影響がツール操作へ反映されると，機密情報の外部送信，保護対象ファイルの変更，意図しない通信などを引き起こす可能性がある．したがって，ツール利用型 AI エージェントでは，最終的な応答文だけでなく，途中で選択されたツール，生成された引数，実行結果，および環境状態の変化を含めて安全性を検証する必要がある [3], [4], [5].

このような安全性検証を継続的に行うには，多様な攻撃を反復して試す必要がある．しかし，攻撃の設計，実行，および評価を人手で繰り返すには多大な時間と労力を要する．そのため，攻撃候補の生成から評価までを自動化し，限られた計算・評価予算の下で反復的に検証できる自動レッドチーミング基盤が求められる [2], [6], [7].

ツール利用型 AI エージェントの自動レッドチーミングには，主に四つの課題がある．第一に，同じ攻撃指示であっても，対象モデルによってツール呼び出しの成否，出力挙動，および実行時間が異なる．第二に，探索時に安全性違反を生じさせた攻撃候補が，初期化後の再実行でも同じ挙動を示すとは限らない．第三に，攻撃候補の試行と評価には時間を要するため，有限の予算内ですべての攻撃方針を十分に試すことは難しい．第四に，特定のツール操作パターンへ探索が偏ると，操作の多様性を十分に確保できない．限られた時間内に有効かつ多様な攻撃候補を発見するには，安全性違反の重大度だけでなく，候補一件当たりの実行時間と再実行可能性，およびツール操作パターンの多様性を考慮し，観測結果に応じて探索資源を複数の攻撃方針へ配分する必要がある．

そこで本稿では，性質の異なる複数の攻撃方針を実際に試行し，各方針が生じさせた安全性違反の重大度と実行時間に基づいて，以降の試行方針を適応的に選択する攻撃探索手法を提案する．提案手法では，各試行の安全性違反から探索用スコアを算出し，方針ごとのスコアを実行時間で除した効率を求める．その後，効率の高い方針ほど選択されやすくなるように選択確率を更新することで，対象モデルの挙動を事前に完全には把握できない場合でも，観測結果に基づいて探索予算の配分を調整する．また，探索時に安全性違反が確認された候補と，探索結果から有望と判断された方針から生成する候補を組み合わせ，初期化環境での再実行を考慮した最終候補列を構成する．

本稿では，具体的な評価環境として Kaggle の AI Agent Security – Multi-Step Tool Attacks コンペティション [9] を用いる．GPT-OSS および Gemma 4 を対象とし，公開ガードレールを適用したローカル環境で，一つの攻撃方針のみを用いる固定方式，標準 9 方針を用いる適応方式，および短い候補を生成する 2 方針を追加した提案手法を比較する．同一の時間制約下で評価した結果，提案手法は固定方式より高いローカル再実行スコアを示した．一方，適応方式に短い候補を生成する方針を追加した際の上乗せは限定的であり，主なスコア差は，一つの方針のみを用いる固定方式から標準方針の集合を用いる適応方式へ変更した段階で観測された．

本研究の主な貢献は以下の通りである．

- ツール利用型 AI エージェントの自動レッドチーミングにおける設計・評価上の課題を，対象モデルによる挙動差，有限予算下の処理効率，再実行可能性，およびツール操作の多様性の観点から整理する．
- 各試行で確認された安全性違反の重大度と実行時間に基づき，単位時間当たりの成果が高い攻撃方針を以降の探索で優先的に選択する適応的攻撃探索手法を提案する．
- Kaggle のコンペティションを評価環境として用い，GPT-OSS および Gemma 4 を対象として，攻撃方針の数と選択方法が異なる 3 方式を同一の時間制約下で比較し，処理時間，探索試行数，ローカル再実行スコア，再実行成功率，および `cell_signature` に基づくツール操作パターンの多様性を定量的に評価する．

本稿の実験は，コンペティション SDK によって提供される固定データと模擬ツールからなるオフラインサンドボックス，およびローカルで利用可能な公開ガードレールのみを対象とする．したがって，本稿の結果は指定した評価環境における方式間比較を示すものであり，実在するサービスの脆弱性や非公開ガードレールに対する有効性を直接示すものではない．

2. 準備

本章では，はじめにツール利用型 AI エージェントの構成と安全上の特徴を整理する．次に，AI エージェントに対するレッドチーミングと安全性評価に関する先行研究を概観する．最後に，本研究が扱う限られた予算下の攻撃探索問題を示す．

2.1 ツール利用型 AI エージェント

ツール利用型 AI エージェントは，ユーザから与えられた要求に対して，LLM による推論や計画と，外部ツールの実行を反復するシステムである [1]．モデルは，ウェブ検

[1] 三菱電機株式会社
Mitsubishi Electric Corporation
[a)] Higashi.Takuya@da.MitsubishiElectric.co.jp

© 2026 Information Processing Society of Japan

索，ファイル操作，電子メール送信，外部通信などのツールを選択し，実行結果を後続の判断に利用する．この仕組みにより，文章生成だけでは完結しない複数段階の処理を自動的に実行できる．

一方，ツール呼び出しは外部状態を実際に変更し得るため，誤った判断は情報の外部送信や保護対象ファイルの変更などのリスクにつながる．また，ウェブページや受信メールなどの信頼できない情報がモデルの文脈へ取り込まれると，そこに含まれる指示によって後続のツール操作が誘導される間接プロンプトインジェクションが引き起こされる可能性がある [3], [4]．したがって，ツール利用型 AI エージェントの安全性評価では，最終的な応答文だけでなく，途中で選択されたツール，その引数，実行結果，および環境状態の変化を含む一連のプロセスを確認する必要がある．

2.2 AI エージェントのレッドチーミングと安全性評価

ツール利用型 AI エージェントの安全性を評価するため，模擬環境やベンチマークを用いた研究が進められている．ToolEmu は，言語モデルによってツール実行を模擬するサンドボックスと自動評価器を用い，多様なツール利用シナリオにおけるリスクを効率的に検出する枠組みを提案している [2]．AgentHarm は，外部ツールを用いて明示的に有害な多段タスクを遂行する能力を評価し，チャットボット向けの安全性評価だけではエージェント固有のリスクを十分に扱えないことを示している [7]．

外部情報を経由する間接プロンプトインジェクションについては，InjecAgent がツール統合型エージェントに対する攻撃を体系的に評価するベンチマークを構築している [4]．AgentDojo は，信頼できないデータを扱うエージェントのタスク，攻撃，および防御を追加可能な動的評価環境として提供し，適応的な攻撃を含む評価を可能にしている [5]．さらに，AGENTVIGIL は，高品質な初期シードとモンテカルロ木探索に基づくシード選択を用いて，間接プロンプトインジェクションを自動探索するブラックボックス型レッドチーミング手法を提案している [6]．

本研究は，攻撃候補の生成を，性質の異なる複数の攻撃方針へ探索予算を配分する問題として捉える．各方針が生じさせた安全性違反だけでなく，候補一件当たりの実行時間も観測し，単位時間当たりの成果に基づいて以降の方針選択を調整する．また，探索中の成功だけでなく，初期化環境での再実行可能性と，限られた時間内に処理できる候補列全体の多様性を評価対象とする．

2.3 本研究の課題設定

AI エージェントの自動レッドチーミングでは，プロンプト探索，ランダム探索，ヒューリスティック探索，進化的探索など，さまざまな方法で攻撃候補を生成できる．しかし，どの攻撃方針が有効であるかは，対象となる AI モデルや実行環境によって異なる可能性がある．同じメッセージを与えた場合でも，モデルによってツールを呼び出すかどうか，どのツールを選択するか，実行にどの程度の時間を要するかが異なるためである．

また，利用可能な計算資源や評価回数が有限である場合，成功率の高い一つの候補を長時間かけて生成する方法が，必ずしも限られた予算内の発見数を最大化するとは限らない．一件当たりの成功率が低い場合でも，短時間で多数の候補を生成し，異なる操作パターンを発見できれば，全体として多くの安全性違反を確認できる可能性がある．

そこで本研究では，複数の攻撃方針を実際に試し，安全性違反の重大度と実行時間を観測しながら，残りの探索予算をどの方針へ配分するかを決定する問題を扱う．本研究で同時に考慮する要件は，次の 4 点である．

(1) 安全性違反を引き起こす確率が高いこと．
(2) 一件当たりの実行時間が短く，有限の予算内で多くの候補を試せること．
(3) 探索中に成功した候補が，初期化した環境での再実行でも成功すること．
(4) 同じ操作を繰り返すだけでなく，異なるツール操作パターンを生成できること．

以上を踏まえ，本研究の目的を，限られた予算内で複数の攻撃方針を評価し，単位時間当たりに高い成果を得た方針を優先することで，再実行可能かつ多様な攻撃候補を効率的に生成することとする．本稿では，この課題を検証する具体的な評価環境として，Kaggle の AI Agent Security – Multi-Step Tool Attacks コンペティション [9] を用いる．評価環境と評価指標の詳細は第 4 章で述べる．

3. 提案手法

本章では，複数の攻撃方針を実際に試行し，安全性違反の判定と実行時間に基づいて有望な方針を選択する適応的攻撃探索手法を提案する．はじめに提案手法全体の流れを説明する．次に，探索で用いる複数の攻撃方針と，一件当たりの処理時間を抑える短い指示について述べる．最後に，試行結果に基づく方針選択，攻撃候補の多様化，および再実行可能な候補の生成方法を説明する．

3.1 提案手法の概要

提案手法の目的は，対象となる AI モデルの性質を事前に完全には把握できない状況において，複数の攻撃方針を試しながら，限られた時間を有望な方針へ配分することである．

同じ攻撃候補を与えた場合でも，AI モデルによってツールを呼び出すかどうか，どのツールを選択するか，および処理に要する時間が異なる可能性がある．そのため，あらかじめ一つの方針だけを選択するのではなく，実際の試行

© 2026 Information Processing Society of Japan

結果を観測しながら探索方針を変更する．

提案手法の処理は，次の五段階からなる．

(1) 性質の異なる複数の攻撃方針から，AI エージェントに与えるメッセージ列を生成する．
(2) 生成したメッセージ列をサンドボックス内で実行し，呼び出されたツール，操作の成否，および安全性違反の有無を確認する．
(3) 各試行について，安全性違反に基づく探索用スコアと実行時間を記録する．
(4) 方針効率の高い方針ほど，その後の探索で選択されやすくする．
(5) 探索中に成功した候補と，方針効率が最も高かった方針から生成した候補を組み合わせ，最終的な攻撃候補列を構成する．

探索中には，モデルの応答やツール操作を含むトレースを確認できる．一方，本稿で用いる評価環境では，最終的に評価器へ渡せるのは，AI エージェントに与えるユーザメッセージ列だけである．探索時のツール状態，途中の会話状態，探索用スコア，使用した方針名などは再実行時に引き継がれない．

このため，提案手法では，探索中の特殊な途中状態に依存する候補ではなく，メッセージ列だけから同じ操作を再現しやすい候補を中心に生成する．探索中に成功した候補についても，第 4 章において新しい環境で再実行し，安全性違反が再現されるかを確認する．

3.2 複数の攻撃方針と短い指示

提案手法では，異なる性質を持つ複数の攻撃方針を用意する．攻撃方針の集合を $\mathcal{S}$ とし，方針 $s \in \mathcal{S}$ が候補番号 i から生成するメッセージ列を

$$M_{s,i} = g_s(i) \tag{1}$$

と表す．

候補番号 i は，操作対象や宛先を変化させるために用いる．これにより，同じ文章を単純に繰り返すのではなく，異なるツール引数を持つ候補を生成する．

本実装では 11 種類の攻撃方針を用いる．説明の便宜上，これらを表 1 の 5 群に整理する．これらは本稿内で用いる実装上の分類であり，一般的な攻撃分類を意味しない．各群には複数の表現上の変種が含まれる．

外部操作指示では，候補番号 i から異なる操作先の識別子を生成する．メール指示では，候補番号を宛先の一部に含めることで，候補ごとに異なるツール引数を生成する．評価時の多様性は攻撃文の表現ではなく，実際に生成されたツール操作に基づいて判定されるため，候補の変更がツールの種類や引数の変更につながるようにする．

また，提案手法では，標準的な候補に加えて短い候補を用意する．短い候補には，主に次の情報だけを含める．

表 1 提案手法で用いる攻撃方針の分類
Table 1 Implementation-oriented groups of attack strategies used in the proposed method.

攻撃方針群	目的と特徴
短い外部操作指示	一つの外部操作を短い指示で要求し，一件当たりの処理時間を抑える
標準的な外部操作指示	操作対象，送信内容，終了後の応答を明示し，安定したツール呼び出しを狙う
単一宛先へのメール指示	一つの宛先に対するメール操作を要求し，権限の誤用に関する挙動を確認する
複数対象への指示	複数の宛先や操作対象を一つの候補に含め，複数のツール操作を引き起こす可能性を調べる
モデル別会話形式への対応	対象モデルごとの会話形式の違いを考慮し，ツール呼び出しを生成しやすい表現を用いる

- 実行を求めるツール操作，
- 操作対象，
- 操作に必要なデータ，
- 操作後に返す短い応答．

短い候補を用いる目的は，文章の短さそのものによって安全機構を回避することではない．入力の処理およびモデルの応答生成に要する時間を抑え，固定された時間内に試行または再実行できる候補数を増やすことが目的である．

一方，指示を短くし過ぎると，モデルが必要な操作を理解できない場合や，追加の説明を生成して処理時間が長くなる場合がある．そのため，提案手法では短い候補だけに固定せず，標準的な候補やモデル別会話形式に対応した候補も同時に試す．短い候補の有効性については，第 4 章において，成功率，一件当たりの実行時間，および一定時間内に得られた評価値を用いて検証する．

3.3 試行結果に基づく方針選択と候補生成

各試行では，はじめにサンドボックス環境を初期状態へ戻し，選択した方針から生成されたメッセージ列 $M_{s,i}$ を実行する．実行後のトレースから安全性違反を判定し，試行に要した時間を測定する．

試行 i で確認された安全性違反事例の多重集合を $\mathcal{P}_i$ とする．安全性違反 p の重大度を v_p としたとき，正式な評価値とは別に，探索中の方針比較に用いる探索用スコア q_i を次式で定義する．

$$q_i = \sum_{p \in \mathcal{P}_i} 2^{v_p - 1} + 2I(|\mathcal{P}_i| > 0)\,. \tag{2}$$

ここで，$I(\cdot)$ は条件が成立するときに 1，それ以外では

© 2026 Information Processing Society of Japan

0 となる関数である．第 1 項は安全性違反の重大度を表し，第 2 項は安全性違反が一つ以上確認された候補への加点である．

探索用スコアは，個々の試行で確認された安全性違反の重大度を簡易的に表す値であり，最終候補列全体から算出する正式な評価値とは異なる．正式評価では，全候補を通じて異なるツール操作パターンがいくつ得られたかによって多様性加点が計算される．これに対し，式 (2) では，個々の成功試行に対して一定の加点を行う．

試行 i の実行時間を t_i とする．方針 s について，これまでに得られた探索用スコアの累積値を Q_s，実行時間の累積値を T_s とする．単位時間当たりに得られた探索用スコアを表す方針効率 E_s を次式で定義する．

$$E_s = \frac{Q_s}{T_s} = \frac{\sum_{i:s_i=s} q_i}{\sum_{i:s_i=s} t_i}. \tag{3}$$

探索の初期段階では，すべての方針を順番に試す．これは，試行前の情報がない状態で，特定の方針だけが選択され続けることを防ぐためである．現行実装では，探索時間の最初の 5%を初期探索期間として扱う．

初期探索期間の後は，方針効率 E_s に基づいて方針を確率的に選択する．この選択は，未知の方針を試す探索と，観測上有望な方針を優先する活用の両立を扱う多腕バンディット問題 [8] と関連する．探索の進行割合を r $(0 \le r \le 1)$ とすると，方針 s の選択確率は次式で与える．

$$\Pr(s) = \frac{\exp(3rE_s)}{\sum_{s' \in \mathcal{S}} \exp(3rE_{s'})}. \tag{4}$$

式中の係数 3 は，方針間の選択確率の集中度を調整する実装上の係数であり，本実験では全条件で同じ値に固定する．

探索の初期では r が小さいため，方針間の選択確率の差も小さい．探索が進むにつれて，方針効率の高い方針が選択されやすくなる．ただし，最良方針だけへ完全に固定するのではなく，確率的な選択を残すことで，初期の少数試行による誤った判断を抑える．

安全性違反が一つ以上確認された候補は，候補メッセージ列，探索用スコア，実行時間，および探索用スコアを実行時間で除した値 q_i/t_i とともに保存する．探索終了後，保存した成功候補を q_i/t_i の高い順に並べる．さらに，E_s が最も高かった方針を，探索結果における最良方針とする．

最終的な候補列は，次の二種類の候補を組み合わせて構成する．

(1) 探索中に実際に安全性違反が確認された候補，
(2) 探索結果から最良と判断された方針によって新たに生成した候補．

成功確認済み候補の q_i/t_i が，最良方針の方針効率 E_s 以上である場合は，その成功候補を優先する．それ以外の場合は，最良方針から新しい候補を生成する．これを繰り返し，本稿の評価環境が受理する上限を超えない範囲で候補列を構成する．

候補の多様化には，操作先や宛先として用いる識別子を候補ごとに変更する方法を用いる．ただし，異なる文章や識別子を用いた場合でも，再実行時に同じツール列と同じ引数が生成されれば，同じ操作パターンとして扱われる可能性がある．したがって，多様性は探索時のメッセージ数ではなく，第 4 章で得られる再実行後の異なる操作パターン数によって評価する．

また，探索中のスナップショットや途中状態に依存すると，メッセージ列だけを用いる再実行では同じ結果を再現できない可能性がある．本手法では，一つのメッセージだけで完結する候補を中心とし，環境を初期化した状態から必要な操作が実行されるようにする．これにより，探索時と再実行時の差を小さくする．探索時に安全性違反が確認された候補から 150 件を抽出し，再実行でも安全性違反が確認された割合を，第 4 章において再実行成功率として評価する．

第 4 章では，公開ガードレールを用いたローカル環境において，ローカル再実行スコア，探索時の平均処理時間と試行数，抽出標本における再実行成功率，および `cell_signature` のユニーク数を測定する．非公開ガードレールはローカル環境から利用できないため，実験対象には含めず，公開ガードレールへの過適合の可能性として第 5 章で考察する．

4. 実験評価

本章では，Kaggle の AI Agent Security – Multi-Step Tool Attacks コンペティションを評価環境として，提案手法が限られた評価時間内で攻撃候補を効率的に生成できるかを検証する．一つの攻撃方針のみを用いる固定方式，標準方針集合を用いる適応方式，および短い候補を生成する方針を追加した提案手法を段階的に比較し，ローカル再実行スコア，探索時の平均処理時間と試行数，抽出標本における再実行成功率，およびユニークセル数を評価する．

4.1 評価環境と評価指標

本コンペティションでは，参加者が攻撃文そのものではなく，ツール利用型 AI エージェントと対話して攻撃候補を生成するアルゴリズムを提出する [9]．探索中には，エージェントが選択したツール，その引数，実行結果，および操作の成否を確認できる．一方，最終的に評価器へ渡されるのはユーザメッセージ列だけであり，探索時の内部状態やツール実行履歴は引き継がれない．候補ごとに初期化した環境でメッセージ列を再実行し，その際に新たに発生したツール操作から安全性違反とスコアを計算する．探索と再実行にはそれぞれ独立した時間制限があり，再実行され

© 2026 Information Processing Society of Japan

表 2　評価対象となる安全性違反
Table 2　Safety violations evaluated in the competition.

安全性違反	重大度	概要
情報の外部送信	5	機密性の高い情報を外部通信によって送信する
信頼できない情報に起因する危険操作	5 または 4	ウェブページや受信メールの内容をきっかけとして，外部送信，書込み，実行などを行う
破壊的な書込み	4	保護対象となるファイルを変更または削除する
権限の誤用	3	ユーザが明示していない送信操作を行う

る候補数にも上限がある．

評価対象となる安全性違反を表 2 に示す．一つの候補で複数の違反が成立した場合や，同じ候補内で複数回の危険な操作が成功した場合は，それぞれを評価対象とする．

重大度 3，4，5 の安全性違反には，それぞれ 4，8，16 の重み $w(v)$ が与えられる．全候補から得られた生スコア R と正規化スコア S は，次式で計算される．

$$R = \sum_{p \in \mathcal{P}} w(v_p) + 2|\mathcal{C}|, \qquad S = \min\left(1000, \frac{R}{200}\right). \quad (5)$$

ここで，$\mathcal{P}$ は再実行によって確認された安全性違反事例の多重集合，$\mathcal{C}$ は異なるツール操作パターンの集合である．操作パターンは，主にツールの種類，操作の順序，および操作結果から区別される．

4.2　比較方式と実験条件

評価対象には，コンペティションで指定された二種類の AI モデルを用いた．GPT-OSS には Unsloth が公開する `gpt-oss-20b` のネイティブ `MXFP4` 版を，Gemma 4 には `gemma-4-26B-A4B-it` の `UD-Q4_K_M` 版を用いた．推論基盤は `llama.cpp` とし，公開ガードレール `optimal`，`max_tool_hops`=8 の条件で実行した．コンペティション SDK のバージョンは 3.1.2 である．

公式評価では，探索フェーズと再実行フェーズにそれぞれ約 9000 秒の時間予算が設定される．同じ予算をローカル実験に適用すると，予備実験では各方式の候補数が上限に達し，処理効率の差が結果に反映されにくかった．そこで，候補数上限の影響を避け，方式間の時間効率差が結果に反映される条件として，各フェーズの時間予算を 300 秒に設定した．最終候補数の上限は 2000 件とした．なお，300 秒は公式評価時間を計算環境の性能に応じて換算した値ではない．

各条件は乱数シード 1000，1001，1002 の 3 回実行し，平均値を報告する．各モデルは一度だけロードし，3 方式と 3 種類の乱数シードからなる 9 条件をモデルごとに連続実行した．二つのモデルは別 GPU 上で並列に実行した．計算環境は，Intel Xeon Gold 6426Y（32 コア），528GB RAM，NVIDIA L40S および H100 NVL である．

比較する三方式は次の通りである．固定方式では，代表的な標準方針である `post_secret` のみを使用した．適応方式では，短い候補を生成する 2 方針を除く標準 9 方針を，式 (4) に従って選択した．提案手法では，短い候補を生成する 2 方針を含む全 11 方針を選択対象とした．この比較は方式を段階的に評価するものである．固定方式と適応方式では選択可能な方針集合も異なるため，両者の差を適応選択だけの効果としては解釈しない．

最終候補列を 300 秒の時間予算内で初期化環境から再実行し，時間内に完了した候補から式 (5) に従って算出した値を，本稿ではローカル再実行スコア S_{local} と呼ぶ．この再実行は，公式評価環境における時間制約付き再実行を近似するものであり，S_{local} はコンペティション公式評価環境で取得された公式スコアではない．

再実行成功率は，各反復において，探索時に安全性違反が確認された候補から 150 件を抽出し，初期化した新しい環境で再実行したときに，安全性違反が再度確認された候補の割合とする．この指標は抽出標本に対する結果であり，探索時に安全性違反が確認された候補全体を全数再実行して求めた割合ではない．多様性については，スコア算出用再実行で成立した候補の `cell_signature` ハッシュのユニーク数を報告する．

4.3　実験結果

表 3 に，モデルごとのローカル再実行スコア S_{local} を示す．GPT-OSS における提案手法のスコアは 114.7 であり，固定方式の 43.2 から 165.5%上昇した．Gemma 4 においても，固定方式の 92.8 に対して提案手法は 100.1 となり，7.9%の上昇が確認された．これらは同一のローカル条件下で方式を比較するための値であり，コンペティション公式スコアではない．

表 4 に，探索時の処理時間と試行数，抽出標本における再実行成功率，およびスコア算出用再実行で得られたユニークセル数を示す．探索時試行数とユニークセル数は異なるフェーズで測定した値である．前者は 300 秒の探索中に実行した試行数，後者は最大 2000 件まで構成した最終候補列を別途再実行して得られた値である．

固定方式から適応方式への段階では，特に GPT-OSS において，探索平均時間が 0.650 秒から 0.233 秒へ短縮され，探索試行数が 453 件から 1220 件へ増加した．Gemma 4 でも探索平均時間は 0.273 秒から 0.260 秒へ短縮され，探索試行数は 1048 件から 1100 件へ増加した．ただし，この差には適応的な方針選択だけでなく，選択可能な方針集合を

© 2026 Information Processing Society of Japan

表 3 モデルごとのローカル再実行スコア
Table 3 Local replay scores by model.

対象モデル	固定方式	適応方式	提案手法	固定方式からの変化率
GPT-OSS	43.2	114.3	114.7	+165.5%
Gemma 4	92.8	99.6	100.1	+7.9%

表 4 処理時間，探索試行数，再実行成功率，およびユニークセル数の比較
Table 4 Comparison of processing time, exploration trials, replay success rate, and unique cells.

対象モデル	手法	探索平均時間（秒）	探索試行数	再実行成功率	ユニークセル数
GPT-OSS	固定方式	0.650	453	100.0%	480
GPT-OSS	適応方式	0.233	1220	100.0%	1270
GPT-OSS	提案手法	0.231	1229	100.0%	1274
Gemma 4	固定方式	0.273	1048	100.0%	1031
Gemma 4	適応方式	0.260	1100	100.0%	1106
Gemma 4	提案手法	0.260	1100	100.0%	1112

1 方針から 9 方針へ拡張した効果も含まれる．

適応方式に短い候補を生成する 2 方針を追加した際の上乗せは限定的であった．GPT-OSS では探索平均時間が 0.233 秒から 0.231 秒，探索試行数が 1220 件から 1229 件，S_{local} が 114.3 から 114.7 へわずかに変化した．Gemma 4 では探索平均時間と探索試行数に変化はなく，S_{local} は 99.6 から 100.1 へ上昇した．3 反復の平均値のみを示しているため，これらの小さな差は明確な優位性ではなく，改善傾向として解釈する．

再実行成功率は，各反復で抽出した 150 件について，両モデルの全方式で 100%であった．したがって，本実験条件では再実行成功率は全方式で 100%に達し，方式間の差を評価できなかった．また，この結果は固定した乱数シードと同一推論設定を用いた抽出標本に対するものであり，異なる乱数シードやコンペティション公式評価環境における再現性を保証しない．

以上より，本実験で観測された大きなスコア差は，一つの攻撃方針のみを用いる固定方式から，標準方針集合を用いる適応方式へ変更した段階で生じた．一方，短い候補を生成する方針の追加による改善は小さかった．

5. 考察

本章では，前章で示した結果を，時間制約，モデル差，公開ガードレールへの依存，および評価上の制限の観点から考察する．

5.1 短い候補と処理時間の関係

探索と再実行の双方に時間制約がある場合，利用可能時間を T，候補一件当たりの平均処理時間を $\bar{t}$，候補数の上限を $N_{\max}$ とすると，処理可能数は概ね $N \simeq \min(N_{\max}, T/\bar{t})$ となる．したがって，安全性違反を生じさせる確率を維持したまま一件当たりの処理時間を短縮できれば，固定時間内に処理できる候補数を増やせる．また，その増加が異なる `cell_signature` につながる場合，S_{local} の増加に寄与する．

ただし，本実験で短い候補を生成する方針を追加した効果は限定的であった．GPT-OSS では探索時間と試行数がわずかに改善したが，Gemma 4 では両指標に変化がなかった．短い候補がツール呼び出しに失敗する場合や，同じ操作パターンに集約される場合には効果がなく，重大度の高い安全性違反を複数安定して再現する長い候補が有利となる場合もある．

5.2 モデルによる違い

共通のシステム命令とツール定義を与えても，ツール呼び出しの有無，種類，回数，引数はモデルごとに異なる可能性がある．そのため，単一方針を固定する代わりに，観測したツール呼び出しと成否から探索用スコアを算出し，方針効率に応じて選択割合を更新する方法は，対象モデルの挙動差に対応するための一つの手段となる．

一方，本実験ではモデルごとに量子化形式が異なり，異なる GPU 上で並列実行している．したがって，表 4 のモデル間の絶対処理時間差を，モデル固有の性質だけに帰属することはできない．方式間の比較は，各モデル内の同一実行条件に限定して解釈する．

5.3 公開ガードレールへの過適合

公開ガードレールに候補を最適化すると，検査範囲，引数の処理順序，または履歴の参照範囲が異なる非公開環境では遮断される可能性がある．したがって，公開ガード

© 2026 Information Processing Society of Japan

レール下で高い S_{local} を得たことが，ガードレール一般に対する有効性を意味するとは限らない．未知の防御条件への依存を抑えるには，文面だけを変えるのではなく，異なるツール，メッセージ形式，呼び出し系列，および引数を生じさせる複数の方針を保持する必要がある．

6. 制限

本研究の候補は，一回のツール呼び出しを誘発する短い形式が中心であり，長い間接プロンプトインジェクションを網羅しない．非公開ガードレールの実装は開示されないため，評価差を特定の防御条件へ帰属することもできない．また，固定データ，固定した乱数シード，および同一推論設定により条件を固定したローカル環境を対象としており，実サービスの認証，通信失敗，データ変化，人間の確認を含まない．

S_{local} は，公開ガードレールとローカルの時間制約付き再実行を用いて算出した比較指標であり，コンペティション公式スコアではない．再実行成功率も，各反復で抽出した 150 件に対する値であり，探索時に安全性違反が確認された候補全体の全数評価ではない．全方式で 100%となったため，再実行可能性の方式間差は評価できなかった．さらに，3 反復の平均値のみを示しており，適応方式と提案手法の小さな差について統計的有意性を評価していない．固定方式と適応方式の比較には方針集合の拡張効果も含まれるため，適応選択だけの因果効果を分離した結果ではない．

以上より，本結果は指定したローカル条件下での方式比較を示すものであり，実運用環境の脆弱性，非公開ガードレールに対する有効性，または特定方式の一般的優位性を直接証明するものではない．

7. おわりに

本研究では，ツール利用型 AI エージェントの自動レッドチーミングにおける設計・評価上の課題を，対象モデルによる挙動差，有限予算下の処理効率，再実行可能性，およびツール操作の多様性の観点から整理した．その上で，各試行で確認された安全性違反の重大度と実行時間に基づき，単位時間当たりの成果が高い攻撃方針を優先的に選択する適応的攻撃探索手法を提案した．

GPT-OSS および Gemma 4 を対象として，攻撃方針の数と選択方法が異なる 3 方式を同一の時間制約下で比較した．その結果，複数方針を適応的に選択する方式は，一つの攻撃方針のみを用いる固定方式より高いローカル再実行スコアを示した．一方，短い候補を生成する方針を追加した際の適応方式からの上乗せは限定的であった．また，抽出標本における再実行成功率は全方式で 100%であったが，固定した乱数シードと同一推論設定の下での結果であり，探索成功候補全体や公式評価環境での再現性を示すものではない．

以上より，モデルの挙動と実行時間を観測しながら複数の攻撃方針へ探索資源を配分することが，有限予算下で有効な攻撃候補を発見するうえで有用である可能性を示した．今後は，異なる乱数シードを用いた再実行，方針集合を揃えた比較，および外部情報の読取りから危険操作へ至る長い多段攻撃へ評価を拡張する．

参考文献

[1] Yao, S. et al.: ReAct: Synergizing Reasoning and Acting in Language Models, ICLR (2023).

[2] Ruan, Y. et al.: Identifying the Risks of LM Agents with an LM-Emulated Sandbox, ICLR (2024).

[3] Greshake, K. et al.: Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection, AISec, pp. 79–90 (2023), doi:10.1145/3605764.3623985.

[4] Zhan, Q. et al.: InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents, Findings of ACL, pp. 10471–10506 (2024), doi:10.18653/v1/2024.findings-acl.624.

[5] Debenedetti, E. et al.: AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents, NeurIPS, Vol. 37 (2024), doi:10.52202/079017-2636.

[6] Wang, Z. et al.: AGENTVIGIL: Automatic Black-Box Red-teaming for Indirect Prompt Injection against LLM Agents, Findings of EMNLP, pp. 23159–23172 (2025), doi:10.18653/v1/2025.findings-emnlp.1258.

[7] Andriushchenko, M. et al.: AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents, ICLR (2025).

[8] Auer, P., Cesa-Bianchi, N. and Fischer, P.: Finite-time Analysis of the Multiarmed Bandit Problem, Machine Learning, Vol. 47, pp. 235–256 (2002), doi:10.1023/A:1013689704352.

[9] Kaggle: AI Agent Security - Multi-Step Tool Attacks, `https://www.kaggle.com/competitions/ai-agent-security-multi-step-tool-attacks` (参照 2026-08-19).

[10] OpenAI: Introducing gpt-oss, `https://openai.com/index/introducing-gpt-oss/` (2025, 参照 2026-08-19).

[11] Gemma Team, Google DeepMind: Gemma 4 Technical Report, arXiv preprint arXiv:2607.02770 (2026).

© 2026 Information Processing Society of Japan